

Reward Design Ablations for GRPO Fine-Tuning in a Controlled LunarLander Benchmark

Kamil Nuriev
Innopolis University
Innopolis, Russia
k.nuriev@innopolis.university

Egor Novikov
Innopolis University
Innopolis, Russia
eg.novikov@innopolis.university

Abstract—We study a controlled LunarLander benchmark implemented as a lightweight adaptation of the SimpleVLA-RL training pipeline. The goal is to test whether reward shaping preserves or degrades a policy that is already competent before applying similar reward-design ideas to more expensive VLA reinforcement learning. All LunarLander suite runs start from the same competent anchor policy and differ only in reward composition. We compare five variants: terminal success only, terminal success with smoothness, terminal success with subgoal and progress shaping, the full combination, and dense shaping without terminal reward. The main finding is that terminal success remains essential. Adding subgoal and progress terms improves final training success while preserving perfect final validation success, whereas smoothness and dense-only shaping degrade the policy despite improving some proxy metrics such as fuel usage.

Index Terms—reinforcement learning, reward shaping, GRPO, LunarLander, policy stability

I. INTRODUCTION

Vision-Language-Action (VLA) models combine visual observations, language instructions, and action prediction in a single policy, making them a natural target for reinforcement learning fine-tuning in robotic manipulation [4]–[6]. SimpleVLA-RL is an RL framework for VLA training built on veRL, with VLA-specific components for trajectory sampling, environment interaction, reward distribution, and policy optimization [6]. The upstream SimpleVLA-RL repository focuses on VLA benchmarks such as LIBERO and RoboTwin and emphasizes binary outcome rewards rather than hand-designed dense reward functions [6].

In this work, we do not attempt to train a full VLA model directly. Instead, we adapt the SimpleVLA-RL-style training pipeline to a lightweight LunarLander setting in the separate LunarLander-GRPO-dense- project. LunarLander is used as a controlled sanity check for reward plumbing and reward-design ablations before applying similar ideas to more expensive VLA training. The goal is not to claim transfer from LunarLander to robotic manipulation, but to answer a smaller question: when GRPO fine-tuning starts from an already competent policy, which reward components preserve or improve performance and which ones destabilize it?

This distinction matters because all runs in the main LunarLander suite start from the same competent anchor check-

point, `lunarlander_baseline_clean_seed42.pt`, rather than from random initialization. The experiment therefore evaluates reward design under policy refinement, not learning from scratch. We focus on the experiment design, the reward ablations, and the empirical outcomes of this controlled sanity check.

II. RELATED WORK

Reward shaping is a common technique for improving credit assignment in reinforcement learning by supplementing sparse terminal rewards with denser intermediate signals [7]. In long-horizon control, such shaping can make learning more stable, but it can also introduce proxy objectives that conflict with the true task goal. This trade-off is central to our study: we test whether subgoal, progress, and smoothness terms preserve or destabilize a policy when terminal success is either retained or removed.

Our optimization setup is related to policy optimization methods such as PPO and GRPO. PPO stabilizes policy-gradient learning through clipped policy updates [1]. GRPO, introduced in DeepSeekMath, replaces a learned critic with group-relative reward normalization, making it attractive for sequence-model fine-tuning when maintaining a separate value model is expensive [2]. DeepSeek-R1 further showed that reinforcement learning with outcome-based rewards can substantially change the behavior of large sequence models [3]. We adopt this group-relative fine-tuning perspective to study how reward composition affects an already competent policy.

Recent VLA reinforcement learning frameworks extend these ideas to robotic policies that map visual observations and language instructions to actions [4]–[6]. SimpleVLA-RL builds on the veRL infrastructure and adapts reinforcement learning to VLA models, including OpenVLA- and OpenVLA-OFT-style policies, with experiments on manipulation benchmarks such as LIBERO and RoboTwin [4]–[6]. Our work is motivated by this setting, but it does not directly train a full VLA model. Instead, we use LunarLander as a lightweight controlled environment for testing reward plumbing and identifying reward-design risks before scaling similar ideas to more expensive VLA training.

Dense reward design for robotic manipulation has also been explored in ReinboT, which motivates reward components such as subgoal achievement, task progress, smoothness, and

TABLE I
REWARD VARIANTS USED IN THE COMPETENT-ANCHOR LUNARLANDER SUITE.

Variant	w_{sub}	w_{prog}	w_{smooth}	w_{final}
terminal_only	0.00	0.00	0.000	1.0
terminal_smooth	0.00	0.00	0.005	1.0
terminal_sub_prog	0.10	0.30	0.000	1.0
full	0.10	0.30	0.005	1.0
dense_only	0.10	0.30	0.005	0.0

terminal success for robot visual-language manipulation [7]. We adapt this high-level decomposition into a simpler LunarLander sanity check. This controlled setting cannot establish transfer to VLA manipulation, but it helps identify which reward components are safe enough to consider before running full VLA-scale ablations.

III. EXPERIMENTAL SETUP

A. Environment and Evaluation Protocol

All LunarLander reward-suite runs use `LunarLander-v3` with the same competent anchor initialization. Validation is performed on a fixed set of 32 seeds with deterministic evaluation and full coverage of all 32 seeds in the final reports. This setup makes the comparison between reward variants controlled: initialization, evaluation seeds, and the training pipeline are held fixed while the reward function changes.

Our adapted LunarLander pipeline logs outcome metrics such as training and validation success, aggregate reward, fuel proxy, and phase-transition diagnostics. The reward code also exposes structured diagnostics over the landing phases ALIGN, DESCEND, and TOUCHDOWN. Although an APPROACH phase exists in the implementation, it was not observed in the competent-anchor suite analyzed here.

B. Reward Variants

The shaped reward used in the suite is

$$R = w_{sub}r_{sub} + w_{prog}(r_{prog} + r_{micro}) + w_{smooth}r_{smooth} + w_{final}r_{final}. \quad (1)$$

Here, r_{final} is a binary terminal success reward, r_{sub} is a phase-local subgoal shaping term, r_{prog} is a one-time phase-entry bonus, r_{micro} is an optional micro-progress bonus, and r_{smooth} penalizes large action switches. In the competent-anchor suite reported here, the micro-progress bonuses were present in the reward interface but did not activate in any run. Therefore, the active comparison is driven by the subgoal, progress, smoothness, and terminal components.

Table I summarizes the five reward variants used in the suite. The `terminal_only` configuration is the control baseline. `terminal_sub_prog` tests whether dense shaping helps when terminal success remains dominant. `full` adds smoothness on top of that setting, and `dense_only` removes terminal success completely.

IV. RESULTS

A. Overall Comparison

Table II reports the main final metrics. The most important result is that `terminal_sub_prog` is the best shaped-reward configuration. It does not exceed `terminal_only` on final validation success because the terminal-only baseline already reaches 1.0000 on the fixed validation seeds. Instead, `terminal_sub_prog` reaches a final training success of 0.9609 while preserving a final validation success of 1.0000, matching the perfect validation score of `terminal_only` and improving substantially over its lower final training success of 0.7422. No reward-hacking warning was triggered in this competent-anchor suite.

By contrast, the other shaped variants degrade the competent anchor. Adding smoothness alone drops final validation success to 0.4375. The `full` reward performs better than `terminal_smooth` but still trails `terminal_sub_prog`, reaching only 0.7812 final validation success. The strongest failure is `dense_only`, which collapses to 0.0234 final training success and 0.0938 final validation success despite having achieved near-perfect intermediate scores earlier in training. This gap between best and final performance indicates that the dense-only objective can actively pull a competent policy away from successful behavior.

B. Effect of Smoothness and Dense Shaping

The comparison between `terminal_sub_prog` and `full` isolates the effect of the smoothness term. Both variants share the same subgoal and progress weights. However, `full` adds smoothness and performs worse on every primary end metric. Final validation success drops from 1.0000 to 0.7812, and final validation reward drops from 1.2347 to 0.7966. Reward-composition diagnostics explain this pattern: in the `full` run, smoothness accounts for a substantial fraction of the total reward, while the terminal-success share falls. Optimization pressure therefore shifts away from the true task objective.

The same pattern appears in the fuel proxy. The variants with smoothness or dense-only shaping use much less control effort, with final validation fuel proxies of 138.50 for `terminal_smooth`, 147.72 for `full`, and 107.06 for `dense_only`, compared with 246.47 for `terminal_only`. However, those lower proxy values coincide with worse landing success. In this suite, reduced fuel use is therefore not a reliable indicator of better policy quality; instead, it is consistent with reduced control activity that does not translate into better landings.

C. Why Terminal Success Must Remain Dominant

The `dense_only` ablation provides the clearest negative result. Even though it still includes subgoal, progress, and smoothness terms, removing the terminal reward almost entirely destroys final performance. The run reaches a best validation success of 0.9688 at one point, but its final validation success is only 0.0938. This behavior shows that dense shaping

TABLE II
MAIN OUTCOME METRICS FOR THE LUNARLANDER REWARD SUITE.

Variant	Final train success	Best train success	Final val success	Best val success	Final val reward	Final val phase transitions
terminal_only	0.7422	0.9609	1.0000	1.0000	1.0000	2.0000
terminal_smooth	0.4922	0.9609	0.4375	0.9688	0.2934	1.5938
terminal_sub_prog	0.9609	0.9766	1.0000	1.0000	1.2347	2.0000
full	0.5547	0.9844	0.7812	0.9688	0.7966	1.7188
dense_only	0.0234	0.9609	0.0938	0.9688	0.0146	1.5312

alone does not anchor optimization to the actual task objective, in this competent-anchor fine-tuning setting.

In contrast, `terminal_sub_prog` preserves terminal success as the dominant component while adding moderate dense shaping. Reward-share diagnostics remain balanced: on validation, the final component still contributes 0.8103 of the total reward, with subgoal and progress contributing 0.0681 and 0.1216, respectively. That combination yields the strongest overall result in the suite: higher final training success than the baseline, no loss in final validation success, and nearly maximal phase progression.

D. Inactive Components and Scope

The phase logs show that the APPROACH state was never used in this competent-anchor suite and that micro-progress did not activate in any run. Consequently, the experiments do not support a claim about whether micro-progress shaping is beneficial or harmful; that component was effectively absent. The active comparison is between terminal reward, subgoal shaping, progress bonuses, and smoothness.

More broadly, these results should be interpreted as an engineering sanity check for reward design and reward plumbing rather than as evidence of transfer to the full VLA setting. The LunarLander path is useful because it exposes reward failures quickly and under controlled conditions, but success here does not by itself establish downstream benefits for vision-language-action training.

V. CONCLUSION

Across the five competent-anchor LunarLander ablations, the strongest shaped reward design is the simple variant that keeps terminal success central while adding only subgoal and progress bonuses. In this benchmark, terminal reward is necessary, smoothness is counterproductive, and dense-only shaping is unstable. The practical recommendation from the current LunarLander suite is therefore to use

$$R = 0.1r_{sub} + 0.3r_{prog} + 1.0r_{final}, \quad (2)$$

with the smoothness term disabled. This configuration improves final training success without sacrificing the perfect final validation success already achieved by the competent anchor and the terminal-only baseline.

VI. PRELIMINARY FULL SIMPLEVLA-RL BASELINE RUN

The full VLA training setting remains substantially more expensive than the LunarLander sanity check. As a separate preliminary feasibility test, we launched a short baseline

SimpleVLA-RL run using an OpenVLA-OFT SFT checkpoint [5], [6]. This run was not a reward-ablation experiment and did not test the LunarLander reward variants in the VLA setting. Its purpose was only to check whether the end-to-end VLA training pipeline could run stably at small scale.

In our preliminary run, training required three A100 GPUs, and attempts with two A100 GPUs resulted in out-of-memory failures. The run used the checkpoint `OpenVLA-OFT-SFT-libero-spatial-trajall` and generated 1200 trajectories per epoch from 10 tasks, 30 environments per task, and 4 trajectories per environment [5]. After filtering environments with all-positive or all-negative trajectory sets, one retained batch contained 48 trajectories. The run completed 12 optimization steps in roughly 9 hours.

Because this run was short and compute-limited, we do not draw conclusions about reward design or downstream VLA performance from it. The observed metrics are treated only as a preliminary stability check. Meaningful comparison of the reward variants studied in LunarLander will require substantially longer VLA runs and a dedicated ablation suite in the full VLA setting.

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [2] Z. Shao *et al.*, “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [3] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [5] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [6] H. Li *et al.*, “SimpleVLA-RL: Scaling VLA training via reinforcement learning,” *arXiv preprint arXiv:2509.09674*, 2025.
- [7] H. Zhang, Z. Zhuang, H. Zhao, P. Ding, H. Lu, and D. Wang, “ReinboT: Amplifying robot visual-language manipulation with reinforcement learning,” *arXiv preprint arXiv:2505.07395*, 2025.